SoSConfigurator

System Design Description

Abstract
Source-based description of the SoSConfigurator support system and its registered services. Documentation edition 5.3.0 retains the implementation and wire versions defined by the current primary source; no runtime upgrade or deployment is implied.

Contents

1 Documentation edition and source scope **3**

2 System context and boundary **3**

3 Functions and external dependencies **3**

4 Configuration and deployment profile **3**

5 State, recovery and limitations **4**

6 Implementation structure **4**

7 Build and verification boundary **4**

8 Security and access model **4**

9 Go and Rust implementation variants **5**

9.1 Go implementation 5

9.2 Rust implementation 5

10 Primary source references **5**

11 Revision history **5**

1 Documentation edition and source scope

This is documentation edition 5.3.0, dated 2026-11-09. No earlier `documentation/v5.2.2` set exists for this system. These documents were written from the primary Java source and secure configuration; no older system documentation was deleted or changed. Runtime/dependency/service versions remain source-defined: `SoSConfiguratorConstants.SYSTEM_VERSION` is 5.2.2 and service versions are 1.0.0. Documentation publication does not upgrade the runtime or deploy an SoS.

2 System context and boundary

SoSConfigurator plans and submits policy-constrained System of Systems configurations spanning Arrowhead local clouds. The catalog defines the organization, relay, eligible member clouds, consumers and exported provider services. An operator requests members and inter-cloud exchanges. A member without exchanges is released from the catalog-scoped inter-cloud relationships; clouds outside the request remain unchanged.

The registered support system produces `sosConfiguration` and `generalManagement`, both service version 1.0.0. The common-library publisher additionally registers/publishes `sysMonitor` when enabled. This current source supersedes the obsolete README paragraph that described an unregistered operations tool.

The operator console and Arrowhead service connector are separate trust boundaries. The service uses the system identity and provider grants. Per-cloud and relay executors carry out ColonyOS work and revalidate requests against their own catalog. SoSConfigurator does not itself provision certificate authorities, relay ACLs or cloud networks.

3 Functions and external dependencies

The API exposes the catalog, observes executor status, creates a plan, submits an allowed plan, lists recent submissions and retrieves a workflow. Planning runs security/authorization rules before it emits relay-check, cloud-check, neighbours, grants, verify and optional discover steps. The required partners are ColonyOS, per-role executors, Gatekeeper/GateTunnel, ConsumerAuthorization, Authentication, ServiceRegistry and optionally the telemetry broker.

The policy engine rejects unsupported inter-cloud implementations, unknown or duplicate members, cross-organization membership, uncatalogued consumers/exports and operations outside an export. Core/management identities and management services are excluded by catalog/policy rules. Executors independently derive their permitted configuration before running the owned cloud/relay scripts.

4 Configuration and deployment profile

The Arrowhead API defaults to port 8494. The operator console defaults to 8495. The standalone secure configuration enables mutual TLS, outsourced authentication, certificate/token binding, Sysop-only general management and operation-specific `sosConfiguration` authorization. It loads identity/store/database/console secrets through `configtree:/run/secrets/` and checks the configured cloud CRL.

The packaged application properties contain an explicit development profile with TLS off and declared authentication; the security guard requires an explicit development-profile choice before it starts that way. These packaged development values do not describe the secure standalone deployment. In the secure container configuration, the console listens on container interfaces but Compose publishes it only on host loopback; the source default console bind is loopback.

The catalog location is `sos.catalog.file`, default `/app/catalogs/sos.json`. Executor status timeout defaults to 20 seconds. `sos.api.whitelist` is empty by default; service authorization defaults to enabled in

`SoSConfiguratorDefaults`. Console sessions default to 28800 seconds. ColonyOS configuration uses `COLONIES_SERVER_HOST`, `COLONIES_SERVER_PORT`, `COLONIES_TLS`, `COLONIES_COLONY_NAME` and `COLONIES_PRVKEY_FILE` (or the supported environment key value).

5 State, recovery and limitations

MySQL schema `ah_sosconfigurator` records the submitter, SoS name, request, permitted plan and submission time; operational logs use the system's own log tables. Workflow state and process logs remain in ColonyOS. The backend submission happens before the local deployment record is saved, so a persistence failure can leave a workflow needing reconciliation. Applying changes across clouds is not a distributed transaction.

A 202 response acknowledges workflow submission, not successful application to every cloud. Observe the resulting workflow, verify the derived grants/neighbours and reconcile partial effects before retrying. Status responses can contain per-executor offline/error objects while the overall HTTP request succeeds. This document task did not execute workflows, apply cloud changes or requalify deployments.

6 Implementation structure

The Java 21/Spring Boot application owns its Maven build and private common-library source. `SoSConfiguratorSystemInfo` registers service definitions and operation paths. `api/http/SosConfigurationAPI.java` parses and serves the application API. `SosConfigurationService` coordinates the planner, ColonyOS adapter and JPA submission repository. `PolicyEngine` and `Planner` implement catalog-based checks and ordering. `ExecutorApplication` and `StepRunner` select the role and owned scripts. `ColonyClient` implements the signed ColonyOS RPC exchange. The console filter/authentication components isolate the browser connector.

A catalog request is read-only. A plan request evaluates the policy without applying configuration. A submission is reevaluated, sent to ColonyOS and then recorded locally. Executors validate their own role/catalog before executing; the planner orders all checks before neighbour/grant mutations, then verifies and optionally discovers the exported services. Cross-system side effects still require reconciliation on failure.

7 Build and verification boundary

Use the system's `pom.xml`, `Makefile` and deployment guide. Maven builds the system from its primary folder and vendored common code. This documentation edition was checked against source annotations, records, service-registration models, filters and configuration; the documents are compiled with the included build script. Historical README test results are not a claim that application or live-cloud tests were rerun for this documentation task.

8 Security and access model

The secure Arrowhead connector uses TLS with client certificates, outsourced identity verification and token-to-certificate binding. `SosAccessFilter` permits a `sosConfiguration` operation for a verified Sysop, a named `sos.api.whitelist` system or a ConsumerAuthorization operation grant. An authenticated identity alone is insufficient. General management remains governed by the common management filter.

The separate console authenticates a browser with a configured scrypt password hash and the `ah_sos_console` cookie. It applies Host/Origin checks and session/lockout rules. A console session does not become

an Arrowhead identity token. Static UI/session paths are console-only; the Arrowhead connector returns 404 for them. Console handling marks authenticated operator requests so the internal API can execute without pretending to be an mTLS system caller.

9 Go and Rust implementation variants

9.1 Go implementation

No `Eclipse-Arrowhead-go/ah5-SoSConfigurator-go` folder exists in the designated primary Go tree. Go local-cloud systems may be targets of a Java SoSConfigurator only if their catalog and inter-cloud implementation meet the policy and executor checks. This is not a Go implementation of SoSConfigurator.

9.2 Rust implementation

No `Eclipse-Arrowhead-rust/ah5-SoSConfigurator-rust` folder exists in the designated primary Rust tree. The policy rejects a catalogued implementation without the required Gatekeeper/GateTunnel inter-cloud capability; the mere presence of Rust local-cloud roles does not establish eligibility or implement this system.

10 Primary source references

All source paths below are relative to `Eclipse-Arrowhead-java/ah5-SoSConfigurator-java`.

- `src/main/java/eu/arrowhead/sosconfigurator/SoSConfiguratorSystemInfo.java` and `SoSConfiguratorConstants.java`: registration, names, versions and paths.
- `api/http/SosConfigurationAPI.java`, `api/http/GeneralManagementAPI.java`, `policy/SosRequest.java`, `catalog/Catalog.java`, `planner/Planner.java` and `service/SosConfigurationService.java`, under the same Java package: operations, DTOs, planning and persistence.
- `security/SosAccessFilter.java` and `console/ConsoleFilter.java` under that package: caller and connector boundaries.
- `config/application.properties`, packaged `src/main/resources/application.properties` and `SoSConfiguratorDefaults.java`: secure overrides versus development/source defaults.
- `vendor/common-utils` and `vendor/data-transfer-objects`: common management, security and native sysMonitor publisher contract.

11 Revision history

2026-11-09: Initial source-based LaTeX documentation edition 5.3.0. Review draft; document compilation and source traceability checked.